ДЗ 4

Марковский Артём Владиславович, 466636, J3310

Репозиторий: https://github.com/shlaapnik/EngData/tree/zookeeper/ZooKeeper

```
import json
import os
import re
import subprocess
import sys
import time
from collections.abc import Callable
from importlib.metadata import version
from pathlib import Path
from typing import Any

import psutil
from kazoo.client import KazooClient
from kazoo.exceptions import NoNodeError

import coordinator

PROJECT = Path.cwd()
LOGS = PROJECT / "logs"
LOGS.mkdir(exist_ok=True)
SERVERS = {"zk1": 2181, "zk2": 2182, "zk3": 2183}
JAVA_INFO = re.compile(r"\d{4}-\d\d-\d\d \d\d:\d\d:\d\d,\d+ \[myid:[^\]]*\] - INFO .*")

s3 = coordinator.s3_client()

print("python", sys.version.split()[0], "kazoo", version("kazoo"), "psutil", version("psutil"))
```

```
python 3.13.7 kazoo 2.11.0 psutil 7.2.2
```

У zkCli убираю только строки INFO из журнала Java, иначе на каждый вызов приходится 40 строк про окружение клиента

```
def record(command: str, output: str) -> None:
    lines = [JAVA_INFO.sub("", line).rstrip() for line in output.splitlines()]
    output = "\n".join(line for line in lines if line != "")
    text = "$ " + command + "\n" + output + "\n"
    print(text)


def sh(*arguments: str) -> None:
    result = subprocess.run(arguments, stdout=subprocess.PIPE, stderr=subprocess.STDOUT, text=True, encoding="utf-8",
                            errors="replace", timeout=120)
    record(" ".join(arguments), result.stdout)


def zk_cli(server: str, command: str) -> None:
    sh("docker", "compose", "exec", "-T", server, "zkCli.sh", "-server", server + ":" + str(SERVERS[server]),
       *command.split())


def mode(server: str) -> str:
    result = subprocess.run(["docker", "compose", "exec", "-T", server, "zkServer.sh", "status"], capture_output=True,
                            text=True, encoding="utf-8", errors="replace")
```

```
    for line in result.stdout.splitlines():
        if line.startswith("Mode:"):
            return line.split()[1]

    return "не отвечает"


def wait_for(check: Callable[[], Any], timeout: float = 900) -> Any:
    deadline = time.monotonic() + timeout

    while time.monotonic() < deadline:
        value = check()

        if value:
            return value

        time.sleep(0.1)

    raise TimeoutError("не дождался за " + str(timeout) + " с")


def show_roles(servers: list[str]) -> dict[str, str]:
    expected = ["follower"] * (len(servers) - 1) + ["leader"]
    wait_for(lambda: sorted(mode(server) for server in servers) == expected, 60)

    for server in servers:
        sh("docker", "compose", "exec", "-T", server, "zkServer.sh", "status")

    return {server: mode(server) for server in servers}
```

1. Подготовка

Поднимаю три сервера ZooKeeper в Docker, Java 17 уже есть в образе. Конфигурации лежат в zk1/conf/zoo.cfg, zk2/conf/zoo.cfg и zk3/conf/zoo.cfg, строки server.N у всех одинаковые, а clientPort, порты кворума и выборов и dataDir у каждого свои, как у трёх копий на одной машине

compose.yaml монтирует каждому серверу его zoo.cfg и отдельный том под dataDir, файл myid в dataDir пишет входной скрипт образа из ZOO_MY_ID, сервер запускается командой zkServer.sh start-foreground. Перед запуском удаляю старые тома, чтобы ансамбль стартовал с пустыми данными

```
sh("docker", "--version")
sh("docker", "compose", "version")
sh("docker", "compose", "down", "-v")
sh("docker", "compose", "up", "-d")
sh("docker", "compose", "exec", "-T", "zk1", "bash", "-c", "echo JAVA_HOME=$JAVA_HOME; java -version")

for server in SERVERS:
    sh("docker", "compose", "exec", "-T", server, "cat", "/conf/zoo.cfg")
    sh("docker", "compose", "exec", "-T", server, "cat", "/data/" + server + "/myid")
```

```
$ docker --version
Docker version 28.5.1, build e180ab8

$ docker compose version
Docker Compose version v2.40.0-desktop.1
```

```
$ docker compose down -v
 Container engdata-zk1  Stopping
 Container engdata-zk2  Stopping
 Container engdata-zk3  Stopping
 Container engdata-zk2  Stopped
 Container engdata-zk2  Removing
 Container engdata-zk1  Stopped
 Container engdata-zk1  Removing
 Container engdata-zk2  Removed
 Container engdata-zk3  Stopped
 Container engdata-zk3  Removing
 Container engdata-zk1  Removed
 Container engdata-zk3  Removed
 Network zookeeper_default  Removing
 Volume zookeeper_zk2-data  Removing
 Volume zookeeper_zk3-data  Removing
 Volume zookeeper_zk1-data  Removing
 Volume zookeeper_zk2-data  Removed
 Volume zookeeper_zk3-data  Removed
 Volume zookeeper_zk1-data  Removed
 Network zookeeper_default  Removed

$ docker compose up -d
 Network zookeeper_default  Creating
 Network zookeeper_default  Created
 Volume zookeeper_zk2-data  Creating
 Volume zookeeper_zk2-data  Created
 Volume zookeeper_zk3-data  Creating
 Volume zookeeper_zk3-data  Created
 Volume zookeeper_zk1-data  Creating
 Volume zookeeper_zk1-data  Created
 Container engdata-zk1  Creating
 Container engdata-zk2  Creating
 Container engdata-zk3  Creating
 Container engdata-zk1  Created
 Container engdata-zk3  Created
 Container engdata-zk2  Created
 Container engdata-zk2  Starting
 Container engdata-zk1  Starting
 Container engdata-zk3  Starting
 Container engdata-zk3  Started
 Container engdata-zk2  Started
 Container engdata-zk1  Started

$ docker compose exec -T zk1 bash -c echo JAVA_HOME=$JAVA_HOME; java -version
JAVA_HOME=/opt/java/openjdk
openjdk version "17.0.21" 2026-10-20
OpenJDK Runtime Environment Temurin-17.0.21+9 (build 17.0.21+9)
OpenJDK 64-Bit Server VM Temurin-17.0.21+9 (build 17.0.21+9, mixed mode, sharing)

$ docker compose exec -T zk1 cat /conf/zoo.cfg
tickTime=2000
initLimit=10
syncLimit=5
dataDir=/data/zk1
clientPort=2181
4lw.commands.whitelist=srvr,mntr,ruok
admin.enableServer=false
server.1=zk1:2888:3888
server.2=zk2:2889:3889
server.3=zk3:2890:3890

$ docker compose exec -T zk1 cat /data/zk1/myid
1
```

```
$ docker compose exec -T zk2 cat /conf/zoo.cfg
tickTime=2000
initLimit=10
syncLimit=5
dataDir=/data/zk2
clientPort=2182
4lw.commands.whitelist=srvr,mntr,ruok
admin.enableServer=false
server.1=zk1:2888:3888
server.2=zk2:2889:3889
server.3=zk3:2890:3890

$ docker compose exec -T zk2 cat /data/zk2/myid
2

$ docker compose exec -T zk3 cat /conf/zoo.cfg
tickTime=2000
initLimit=10
syncLimit=5
dataDir=/data/zk3
clientPort=2183
4lw.commands.whitelist=srvr,mntr,ruok
admin.enableServer=false
server.1=zk1:2888:3888
server.2=zk2:2889:3889
server.3=zk3:2890:3890

$ docker compose exec -T zk3 cat /data/zk3/myid
3
```

Java 17 из образа, у трёх серверов одинаковые строки server.N, а clientPort и dataDir свои, myid совпадает с номером сервера

2. Ансамбль ZooKeeper

Роли смотрю командой zkServer.sh status, запись и чтение проверяю через zkCli

```
roles = show_roles(list(SERVERS))
sh("docker", "compose", "ps")
print("роли:", roles)
```

```
$ docker compose exec -T zk1 zkServer.sh status
ZooKeeper JMX enabled by default
Using config: /conf/zoo.cfg
Client port found: 2181. Client address: localhost. Client SSL: false.
Mode: follower

$ docker compose exec -T zk2 zkServer.sh status
ZooKeeper JMX enabled by default
Using config: /conf/zoo.cfg
Client port found: 2182. Client address: localhost. Client SSL: false.
Mode: follower

$ docker compose exec -T zk3 zkServer.sh status
ZooKeeper JMX enabled by default
Using config: /conf/zoo.cfg
Client port found: 2183. Client address: localhost. Client SSL: false.
Mode: leader
```

```
$ docker compose ps
NAME          IMAGE             COMMAND                  SERVICE   CREATED          STATUS          PORTS
engdata-zk1   zookeeper:3.9.6   "/docker-entrypoint.…"   zk1       18 seconds ago   Up 17 seconds   0.0.0.0:2181->2181/tcp, [::]:2181->2181/tcp
engdata-zk2   zookeeper:3.9.6   "/docker-entrypoint.…"   zk2       18 seconds ago   Up 17 seconds   0.0.0.0:2182->2182/tcp, [::]:2182->2182/tcp
engdata-zk3   zookeeper:3.9.6   "/docker-entrypoint.…"   zk3       18 seconds ago   Up 17 seconds   0.0.0.0:2183->2183/tcp, [::]:2183->2183/tcp

роли: {'zk1': 'follower', 'zk2': 'follower', 'zk3': 'leader'}
```

Останавливаю лидера, двум оставшимся серверам хватает кворума, они выбирают нового лидера, один пишет узел, другой его читает

```
first = [server for server in SERVERS if roles[server] == "leader"][0]
sh("docker", "compose", "stop", first)
alive = [server for server in SERVERS if server != first]
roles = show_roles(alive)
print("роли:", roles)

zk_cli(alive[0], "create /hw4-check written-by-two-servers")
zk_cli(alive[1], "get /hw4-check")
```

```
$ docker compose stop zk3
 Container engdata-zk3  Stopping
 Container engdata-zk3  Stopped

$ docker compose exec -T zk1 zkServer.sh status
ZooKeeper JMX enabled by default
Using config: /conf/zoo.cfg
Client port found: 2181. Client address: localhost. Client SSL: false.
Mode: follower

$ docker compose exec -T zk2 zkServer.sh status
ZooKeeper JMX enabled by default
Using config: /conf/zoo.cfg
Client port found: 2182. Client address: localhost. Client SSL: false.
Mode: leader

роли: {'zk1': 'follower', 'zk2': 'leader'}
$ docker compose exec -T zk1 zkCli.sh -server zk1:2181 create /hw4-check written-by-two-servers
Connecting to zk1:2181
WATCHER::
WatchedEvent state:SyncConnected type:None path:null zxid: -1
Created /hw4-check

$ docker compose exec -T zk2 zkCli.sh -server zk2:2182 get /hw4-check
Connecting to zk2:2182
WATCHER::
WatchedEvent state:SyncConnected type:None path:null zxid: -1
written-by-two-servers
```

Без zk3 оставшиеся zk1 и zk2 выбрали лидером zk2, запись через zk1 и чтение через zk2 прошли

Останавливаю второй сервер, последнему не хватает голосов, он выходит из роли и перестаёт отвечать

```
second = [server for server in alive if roles[server] == "follower"][0]
lone = [server for server in alive if server != second][0]
sh("docker", "compose", "stop", second)
wait_for(lambda: mode(lone) == "не отвечает", 60)

sh("docker", "compose", "ps", "-a")
sh("docker", "compose", "exec", "-T", lone, "zkServer.sh", "status")
sh("docker", "compose", "exec", "-T", lone, "bash", "-c", "echo srvr | nc localhost " + str(SERVERS[lone]))
zk_cli(lone, "get /hw4-check")
```

```
$ docker compose stop zk1
 Container engdata-zk1  Stopping
 Container engdata-zk1  Stopped

$ docker compose ps -a
NAME          IMAGE             COMMAND                  SERVICE   CREATED          STATUS                       PORTS
engdata-zk1   zookeeper:3.9.6   "/docker-entrypoint.…"   zk1       32 seconds ago   Exited (143) 2 seconds ago
engdata-zk2   zookeeper:3.9.6   "/docker-entrypoint.…"   zk2       32 seconds ago   Up 31 seconds                0.0.0.0:2182->2182/tcp, [::]:2182->2182/tcp
engdata-zk3   zookeeper:3.9.6   "/docker-entrypoint.…"   zk3       32 seconds ago   Exited (143) 13 seconds ago

$ docker compose exec -T zk2 zkServer.sh status
ZooKeeper JMX enabled by default
Using config: /conf/zoo.cfg
Client port found: 2182. Client address: localhost. Client SSL: false.
Error contacting service. It is probably not running.

$ docker compose exec -T zk2 bash -c echo srvr | nc localhost 2182
This ZooKeeper instance is not currently serving requests

$ docker compose exec -T zk2 zkCli.sh -server zk2:2182 get /hw4-check
Connecting to zk2:2182
2026-11-09 15:41:49,763 [myid:zk2:2182] - WARN  [main-SendThread(zk2:2182):o.a.z.ClientCnxn$SendThread@1301] - Session 0x0 for server zk2/172.26.0.3:2182, Closing socket connection. Attempting reconnect except it is a SessionExpi
redException or SessionTimeoutException.
org.apache.zookeeper.ClientCnxn$EndOfStreamException: Unable to read additional data from server sessionid 0x0, likely server has closed socket
        at org.apache.zookeeper.ClientCnxnSocketNIO.doIO(ClientCnxnSocketNIO.java:77)
        at org.apache.zookeeper.ClientCnxnSocketNIO.doTransport(ClientCnxnSocketNIO.java:356)
        at org.apache.zookeeper.ClientCnxn$SendThread.run(ClientCnxn.java:1290)
KeeperErrorCode = ConnectionLoss for /hw4-check
2026-11-09 15:41:49,876 [myid:] - ERROR [main:o.a.z.u.ServiceUtils@48] - Exiting JVM with code 1
```

Один zk2 кворум не собирает, zkServer.sh status не может связаться с сервисом, на srvr сервер отвечает not currently serving requests, zkCli получает ConnectionLoss

Запускаю оба остановленных сервера снова

```
sh("docker", "compose", "start", first, second)
roles = show_roles(list(SERVERS))
print("роли:", roles)
zk_cli(first, "get /hw4-check")
```

```
$ docker compose start zk3 zk1
 Container engdata-zk1  Starting
 Container engdata-zk3  Starting
 Container engdata-zk1  Started
 Container engdata-zk3  Started

$ docker compose exec -T zk1 zkServer.sh status
ZooKeeper JMX enabled by default
Using config: /conf/zoo.cfg
Client port found: 2181. Client address: localhost. Client SSL: false.
Mode: follower

$ docker compose exec -T zk2 zkServer.sh status
ZooKeeper JMX enabled by default
Using config: /conf/zoo.cfg
Client port found: 2182. Client address: localhost. Client SSL: false.
Mode: leader

$ docker compose exec -T zk3 zkServer.sh status
ZooKeeper JMX enabled by default
Using config: /conf/zoo.cfg
Client port found: 2183. Client address: localhost. Client SSL: false.
Mode: follower

роли: {'zk1': 'follower', 'zk2': 'leader', 'zk3': 'follower'}
```

```
$ docker compose exec -T zk3 zkCli.sh -server zk3:2183 get /hw4-check
Connecting to zk3:2183
WATCHER::
WatchedEvent state:SyncConnected type:None path:null zxid: -1
written-by-two-servers
```

После возврата zk1 и zk3 стали последователями, лидером остался zk2, узел /hw4-check читается с zk3, который был выключен во время записи и получил узел при возвращении

Эфемерный узел. Второе окно это отдельный процесс zkCli, ему на вход подаю create -e, закрыть окно значит закрыть его вход, как при закрытии терминала. Из первого окна смотрю узел и его владельца, потом жду, пока узел пропадёт

```
window_command = "docker compose exec -T zk1 zkCli.sh -server zk1:2181"
window = subprocess.Popen(window_command.split(), stdin=subprocess.PIPE, stdout=subprocess.PIPE,
                          stderr=subprocess.STDOUT, text=True, encoding="utf-8", errors="replace")
window.stdin.write("create -e /hw4-ephemeral window-2\n")
window.stdin.flush()

zk = KazooClient(hosts=coordinator.HOSTS, timeout=coordinator.SESSION_TIMEOUT)
zk.start()
wait_for(lambda: zk.exists("/hw4-ephemeral"), 60)
zk_cli("zk2", "stat /hw4-ephemeral")

window.stdin.close()
closed = time.monotonic()
record(window_command + ", второе окно, на входе create -e /hw4-ephemeral window-2", window.communicate(timeout=60)[0])
zk_cli("zk2", "ls /")

wait_for(lambda: not zk.exists("/hw4-ephemeral"), 90)
print("узел пропал через", round(time.monotonic() - closed, 1), "с после закрытия окна")
zk_cli("zk2", "ls /")
```

```
$ docker compose exec -T zk2 zkCli.sh -server zk2:2182 stat /hw4-ephemeral
Connecting to zk2:2182
WATCHER::
WatchedEvent state:SyncConnected type:None path:null zxid: -1
cZxid = 0x300000004
ctime = Mon Nov 09 15:42:05 UTC 2026
mZxid = 0x300000004
mtime = Mon Nov 09 15:42:05 UTC 2026
pZxid = 0x300000004
cversion = 0
dataVersion = 0
aclVersion = 0
ephemeralOwner = 0x100003480bf0001
dataLength = 8
numChildren = 0

$ docker compose exec -T zk1 zkCli.sh -server zk1:2181, второе окно, на входе create -e /hw4-ephemeral window-2
Connecting to zk1:2181
Welcome to ZooKeeper!
JLine support is enabled
WATCHER::
WatchedEvent state:SyncConnected type:None path:null zxid: -1
[zk: zk1:2181(CONNECTED) 0] Created /hw4-ephemeral
[zk: zk1:2181(CONNECTED) 1] [zk: zk1:2181(CONNECTED) 1]

$ docker compose exec -T zk2 zkCli.sh -server zk2:2182 ls /
Connecting to zk2:2182
WATCHER::
WatchedEvent state:SyncConnected type:None path:null zxid: -1
[hw4-check, hw4-ephemeral, zookeeper]

узел пропал через 30.5 с после закрытия окна
```

```
$ docker compose exec -T zk2 zkCli.sh -server zk2:2182 ls /
Connecting to zk2:2182
WATCHER::
WatchedEvent state:SyncConnected type:None path:null zxid: -1
[hw4-check, zookeeper]
```

Пока окно было открыто, у узла был владелец ephemeralOwner, это сессия второго окна. Закрытие окна сессию не завершило, она истекла по тайм-ауту zkCli в 30 с, и через 30.5 с ZooKeeper удалил узел

3. Приостановка лидера без защиты

coordinator.py это координатор расчёта пар навыков из ДЗ 3, map и reduce он берёт из mapreduce_job.py. Обе копии подключаются к ансамблю с тайм-аутом сессии 10 с и встают в выборы через Election из kazoo. Победитель увеличивает узел epoch, его версия и есть эпоха, и создаёт эфемерный узел leader

Чекпоинт это выходы готовых map-задач в RustFS, в checkpoints/hw4/<запуск>/, а прогресс лидер пишет в узел checkpoint, поэтому запасной продолжает с того места, где остановился лидер. Результат лидер кладёт в RustFS отдельным CSV, а в узел result пишет указатель на этот файл

Лидера вместе с его процессами-воркерами приостанавливаю через psutil, когда в чекпоинте 320 из 373 map-задач, и возобновляю, когда запасной записал результат. Время смены лидера считаю от приостановки до появления нового узла leader, перед каждым запуском перезапускаю RustFS

```python
SUSPEND_AT = 320


def fresh_start() -> None:
    subprocess.run(["docker", "restart", "rustfs"], capture_output=True)

    for attempt in range(90):
        try:
            coordinator.read_pairs(s3, coordinator.HW3_PAIRS)
            break
        except Exception:
            time.sleep(2)

    time.sleep(5)


def read_node(run: str, part: str) -> dict[str, Any]:
    try:
        return json.loads(zk.get(coordinator.node(run, part))[0])
    except NoNodeError:
        return {}


def reset_run(run: str) -> None:
    if zk.exists(coordinator.ROOT + "/" + run):
        zk.delete(coordinator.ROOT + "/" + run, recursive=True)

    zk.create(coordinator.node(run, "epoch"), b"0", makepath=True)
    zk.create(coordinator.node(run, "checkpoint"), b"{}")
    zk.create(coordinator.node(run, "result"), b"{}")

    for prefix in (coordinator.checkpoint_prefix(run), coordinator.result_prefix(run)):
        for page in s3.get_paginator("list_objects_v2").paginate(Bucket=coordinator.BUCKET, Prefix=prefix):
```

```
            for item in page.get("Contents", []):
                s3.delete_object(Bucket=coordinator.BUCKET, Key=item["Key"])

    for path in (LOGS / run).glob("*.log"):
        path.unlink()


def failover_run(run: str, fencing: bool) -> dict[str, float]:
    fresh_start()
    reset_run(run)
    environment = dict(os.environ, FENCING="1" if fencing else "0")
    processes = []

    for name in ("coordinator-a", "coordinator-b"):
        processes.append(subprocess.Popen([sys.executable, "coordinator.py", run, name], env=environment,
                                          stdout=subprocess.DEVNULL, stderr=subprocess.DEVNULL))
        time.sleep(1)

    leader = wait_for(lambda: read_node(run, "leader"))
    print("лидер", leader["coordinator"], "эпоха", leader["epoch"])

    def ready() -> dict[str, Any]:
        checkpoint = read_node(run, "checkpoint")

        return checkpoint if checkpoint.get("done", 0) >= SUSPEND_AT else {}

    def new_leader() -> dict[str, Any]:
        current = read_node(run, "leader")

        return current if current.get("coordinator", leader["coordinator"]) != leader["coordinator"] else {}

    checkpoint = wait_for(ready)
    parent = psutil.Process(leader["pid"])
    paused = [parent] + parent.children(recursive=True)

    for process in paused:
        process.suspend()

    paused_at = time.monotonic()
    print("приостановил", leader["coordinator"], "и", len(paused) - 1, "его процессов на чекпоинте", checkpoint["done"],
          "из", checkpoint["total"])

    backup = wait_for(new_leader)
    failover = round(time.monotonic() - paused_at, 1)
    print("новый лидер", backup["coordinator"], "эпоха", backup["epoch"], "через", failover, "с после приостановки")

    wait_for(lambda: read_node(run, "result").get("coordinator") == backup["coordinator"])

    for process in paused:
        process.resume()

    pause = round(time.monotonic() - paused_at, 1)
    print(backup["coordinator"], "записал результат, возобновил", leader["coordinator"], "через", pause, "с после приостановки")

    for process in processes:
        process.wait()

    data, stat = zk.get(coordinator.node(run, "result"))
    print("указатель", data.decode(), "версия", stat.version)

    for name in ("coordinator-a", "coordinator-b"):
        print()
        print("logs/" + run + "/" + name + ".log")
        print((LOGS / run / (name + ".log")).read_text(encoding="utf-8"))

    return {"failover": failover, "pause": pause, "pointer_version": stat.version}
```

```
plain = failover_run("no-fencing", fencing=False)
```

```
лидер coordinator-a эпоха 1
приостановил coordinator-a и 12 его процессов на чекпоинте 323 из 373
новый лидер coordinator-b эпоха 2 через 12.7 с после приостановки
```

```
coordinator-b записал результат, возобновил coordinator-a через 226.6 с после приостановки
указатель {"key": "results/hw4/no-fencing/coordinator-a-epoch1.csv", "coordinator": "coordinator-a", "epoch": 1} версия 2

logs/no-fencing/coordinator-a.log
18:07:28.542 coordinator-a сессия ZooKeeper активна
18:07:28.543 coordinator-a запущен, pid 24660, тайм-аут сессии 10.0 с, защита по эпохе выключена
18:07:28.543 coordinator-a жду выборов
18:07:29.487 coordinator-a стал лидером, эпоха 1
18:07:30.327 coordinator-a начинаю расчёт, map-задач 373
18:08:29.837 coordinator-a чекпоинт: готово 26 из 373
18:09:27.855 coordinator-a чекпоинт: готово 54 из 373
18:10:26.235 coordinator-a чекпоинт: готово 82 из 373
18:11:16.485 coordinator-a чекпоинт: готово 113 из 373
18:11:49.613 coordinator-a чекпоинт: готово 141 из 373
18:12:24.469 coordinator-a чекпоинт: готово 169 из 373
18:12:57.581 coordinator-a чекпоинт: готово 197 из 373
18:13:46.574 coordinator-a чекпоинт: готово 225 из 373
18:14:31.828 coordinator-a чекпоинт: готово 253 из 373
18:15:36.372 coordinator-a чекпоинт: готово 281 из 373
18:16:27.092 coordinator-a чекпоинт: готово 309 из 373
18:20:45.397 coordinator-a kazoo: Connection dropped: socket connection broken
18:20:45.422 coordinator-a kazoo: Transition to CONNECTING
18:20:45.427 coordinator-a связь с ZooKeeper потеряна
18:20:45.531 coordinator-a kazoo: Session has expired
18:20:45.531 coordinator-a сессия ZooKeeper истекла
18:20:45.830 coordinator-a сессия ZooKeeper активна
18:21:05.651 coordinator-a чекпоинт: готово 373 из 373
18:21:33.674 coordinator-a расчёт закончен за 844.2 с, уникальных пар 10999, сумма частот 48945, совпадает с ДЗ 3: да
18:21:34.434 coordinator-a записал результат results/hw4/no-fencing/coordinator-a-epoch1.csv, версия указателя 2
18:21:34.488 coordinator-a остановлен


logs/no-fencing/coordinator-b.log
18:07:29.314 coordinator-b сессия ZooKeeper активна
18:07:29.314 coordinator-b запущен, pid 1556, тайм-аут сессии 10.0 с, защита по эпохе выключена
18:07:29.314 coordinator-b жду выборов
18:17:11.361 coordinator-b стал лидером, эпоха 2
18:17:21.382 coordinator-b продолжаю с чекпоинта: готово 324 из 373 map-задач
18:18:56.450 coordinator-b чекпоинт: готово 352 из 373
18:20:31.331 coordinator-b чекпоинт: готово 373 из 373
18:20:43.945 coordinator-b расчёт закончен за 212.6 с, уникальных пар 10999, сумма частот 48945, совпадает с ДЗ 3: да
18:20:45.155 coordinator-b записал результат results/hw4/no-fencing/coordinator-b-epoch2.csv, версия указателя 1
18:20:45.503 coordinator-b остановлен
```

Лидер сменился через 12.7 с после приостановки, это 10 с тайм-аута сессии и выборы с подъёмом эпохи. Запасной продолжил с 324 готовых map-задач из 373 и получил результат ДЗ 3, а проснувшийся coordinator-a тоже записал результат, в журналах две строки с результатом, указатель версии 2 ведёт на файл старой эпохи 1

4. Приостановка лидера с защитой по эпохе

С FENCING=1 координатор пишет указатель транзакцией из двух операций: check версии узла epoch и set_data узла result. Пока лидер стоял, запасной поднял эпоху, поэтому check старого лидера не проходит и ZooKeeper отклоняет всю транзакцию

```
fenced = failover_run("fencing", fencing=True)
```

```
лидер coordinator-a эпоха 1
приостановил coordinator-a и 12 его процессов на чекпоинте 326 из 373
новый лидер coordinator-b эпоха 2 через 12.9 с после приостановки
coordinator-b записал результат, возобновил coordinator-a через 176.5 с после приостановки
```

```
указатель {"key": "results/hw4/fencing/coordinator-b-epoch2.csv", "coordinator": "coordinator-b", "epoch": 2} версия 1

logs/fencing/coordinator-a.log
18:21:52.342 coordinator-a сессия ZooKeeper активна
18:21:52.342 coordinator-a запущен, pid 26524, тайм-аут сессии 10.0 с, защита по эпохе включена
18:21:52.342 coordinator-a жду выборов
18:21:53.458 coordinator-a стал лидером, эпоха 1
18:21:54.353 coordinator-a начинаю расчёт, map-задач 373
18:23:10.971 coordinator-a чекпоинт: готово 27 из 373
18:24:15.017 coordinator-a чекпоинт: готово 55 из 373
18:25:12.410 coordinator-a чекпоинт: готово 83 из 373
18:26:11.267 coordinator-a чекпоинт: готово 111 из 373
18:27:01.333 coordinator-a чекпоинт: готово 137 из 373
18:27:56.728 coordinator-a чекпоинт: готово 165 из 373
18:28:56.850 coordinator-a чекпоинт: готово 193 из 373
18:29:59.187 coordinator-a чекпоинт: готово 221 из 373
18:31:01.293 coordinator-a чекпоинт: готово 249 из 373
18:32:23.894 coordinator-a чекпоинт: готово 277 из 373
18:33:48.056 coordinator-a чекпоинт: готово 305 из 373
18:37:36.552 coordinator-a kazoo: Connection dropped: socket connection broken
18:37:36.598 coordinator-a kazoo: Transition to CONNECTING
18:37:36.602 coordinator-a связь с ZooKeeper потеряна
18:37:36.777 coordinator-a kazoo: Session has expired
18:37:36.777 coordinator-a сессия ZooKeeper истекла
18:37:36.897 coordinator-a сессия ZooKeeper активна
18:37:54.068 coordinator-a чекпоинт: готово 373 из 373
18:38:19.563 coordinator-a расчёт закончен за 986.1 с, уникальных пар 10999, сумма частот 48945, совпадает с ДЗ 3: да
18:38:20.388 coordinator-a запись результата отклонена: BadVersionError, моя эпоха 1, текущая 2
18:38:20.417 coordinator-a остановлен


logs/fencing/coordinator-b.log
18:21:52.728 coordinator-b сессия ZooKeeper активна
18:21:52.728 coordinator-b запущен, pid 25772, тайм-аут сессии 10.0 с, защита по эпохе включена
18:21:52.728 coordinator-b жду выборов
18:34:52.913 coordinator-b стал лидером, эпоха 2
18:35:02.853 coordinator-b продолжаю с чекпоинта: готово 327 из 373 map-задач
18:36:22.849 coordinator-b чекпоинт: готово 355 из 373
18:37:24.661 coordinator-b чекпоинт: готово 373 из 373
18:37:35.421 coordinator-b расчёт закончен за 162.5 с, уникальных пар 10999, сумма частот 48945, совпадает с ДЗ 3: да
18:37:36.519 coordinator-b записал результат results/hw4/fencing/coordinator-b-epoch2.csv, версия указателя 1
18:37:36.553 coordinator-b остановлен
```

Лидер сменился через 12.9 с, запасной продолжил с 327 из 373. Старый лидер досчитал и попытался записать указатель с эпохой 1, транзакция вернула BadVersionError, указатель остался версии 1 от coordinator-b с эпохой 2

Сверка с ДЗ 3: каждый CSV, который координаторы положили в RustFS, сравниваю с results/mapreduce/skills_k2.csv из ДЗ 3 по числу пар, сумме частот и контрольному хешу

```
hw3 = coordinator.read_pairs(s3, coordinator.HW3_PAIRS)
hw3_hash = coordinator.canonical_hash(hw3)
print(coordinator.HW3_PAIRS, "пар", len(hw3), "сумма частот", sum(count for _, count in hw3), "sha256", hw3_hash[:16])
print()

for run in ("no-fencing", "fencing"):
    pointer = read_node(run, "result")

    for page in s3.get_paginator("list_objects_v2").paginate(Bucket=coordinator.BUCKET, Prefix=coordinator.result_prefix(run)):
        for item in page.get("Contents", []):
            pairs = coordinator.read_pairs(s3, item["Key"])
            same = coordinator.canonical_hash(pairs) == hw3_hash
            print(item["Key"], "пар", len(pairs), "сумма частот", sum(count for _, count in pairs), "совпадает с ДЗ 3:", same,
                  "на него указывает result:", item["Key"] == pointer["key"])
```

```
results/mapreduce/skills_k2.csv пар 10999 сумма частот 48945 sha256 cf6a0b45843b4de5

results/hw4/no-fencing/coordinator-a-epoch1.csv пар 10999 сумма частот 48945 совпадает с ДЗ 3: True на него указывает result: True
results/hw4/no-fencing/coordinator-b-epoch2.csv пар 10999 сумма частот 48945 совпадает с ДЗ 3: True на него указывает result: False
results/hw4/fencing/coordinator-a-epoch1.csv пар 10999 сумма частот 48945 совпадает с ДЗ 3: True на него указывает result: False
results/hw4/fencing/coordinator-b-epoch2.csv пар 10999 сумма частот 48945 совпадает с ДЗ 3: True на него указывает result: True
```

Все четыре CSV совпали с ДЗ 3. Без защиты result ведёт на файл старого лидера с эпохой 1, с защитой на файл запасного с эпохой 2

5. Ответы

Почему серверов три, а не два и не четыре

ZooKeeper принимает запись, когда её подтвердило строгое большинство серверов, больше половины. Любые два таких большинства пересекаются хотя бы в одном сервере, поэтому две части кластера не могут одновременно выбрать двух лидеров

У трёх серверов большинство это два, один сервер можно потерять, это видно в разделе 2. У двух серверов большинство тоже два, отказ любого из них останавливает запись, то есть второй сервер добавляет ещё одну точку отказа и ничего не даёт взамен. У четырёх большинство три, они переживают тот же один отказ, что и три, но каждую запись ждут от трёх серверов, а при разрыве сети 2 на 2 ни одна половина не наберёт большинства. Поэтому серверов берут нечётное число, и три это наименьшее, которое переживает отказ

Почему защиту по эпохе проверяет получатель записи, а не ZooKeeper

Лидерство держится на сессии в ZooKeeper, а приостановленный процесс не знает, что его сессия истекла. В разделе 3 старый лидер проснулся, досчитал и записал результат с эпохой 1, хотя лидером уже был другой координатор. Проверка "я ещё лидер?" перед записью тоже не спасает, процесс может встать на паузу сразу после неё, и запись уйдёт уже устаревшей

ZooKeeper знает про сессии и выборы, а саму запись в хранилище он не видит. Отклонить устаревшего писателя может только тот, кто принимает запись, в момент её применения, он сравнивает эпоху из записи с текущей и отказывает меньшей одной атомарной операцией. Здесь получатель это узел result, поэтому проверка сделана транзакцией check(epoch) и set_data(result), если бы результат шёл в базу или в S3, сравнивать эпоху пришлось бы им

```
sh(sys.executable, "-m", "nbconvert", "--to", "html", "zookeeper.ipynb")
PRINT_STYLE = """<style>
@page { size: A4 landscape; margin: 8mm; }
@media print {
  .jp-InputPrompt, .jp-OutputPrompt { display: none !important; }
  .jp-OutputArea-output, .jp-OutputArea-child, .jp-Cell-outputWrapper, .jp-Cell-inputWrapper, .jp-InputArea-editor { overflow: visible !important; }
  .jp-RenderedHTMLCommon table { font-size: 8px !important; table-layout: auto !important; }
  .jp-RenderedHTMLCommon td, .jp-RenderedHTMLCommon th { padding: 1px 3px !important; }
  .jp-RenderedText pre, .jp-InputArea-editor pre, .highlight pre { font-size: 8px !important; line-height: 1.25 !important; white-space: pre-wrap !important; word-break: break-all !important; }
  .jp-RenderedImage img { max-width: 100% !important; }
}
</style>
```

```
"""
report = PROJECT / "zookeeper.html"
report.write_text(report.read_text(encoding="utf-8").replace("</head>", PRINT_STYLE + "</head>", 1), encoding="utf-8")
print("html", report.stat().st_size, "байт")
```

```
$ C:\python\Study\ITMO_3nd\EngData\ZooKeeper\.venv\Scripts\python.exe -m nbconvert --to html zookeeper.ipynb
[NbConvertApp] Converting notebook zookeeper.ipynb to html
[NbConvertApp] Writing 382767 bytes to zookeeper.html

html 397971 байт
```